

UNIVERSIDAD CARLOS III



Sistemas Distribuidos

Ejercicio Evaluable 3: RPC

CURSO 2025 - 2026

XIYA YE → 100522159@alumnos.uc3m.es

Grupo 85, Ingeniería Informática

WENJIE CHEN → 100522255@alumnos.uc3m.es

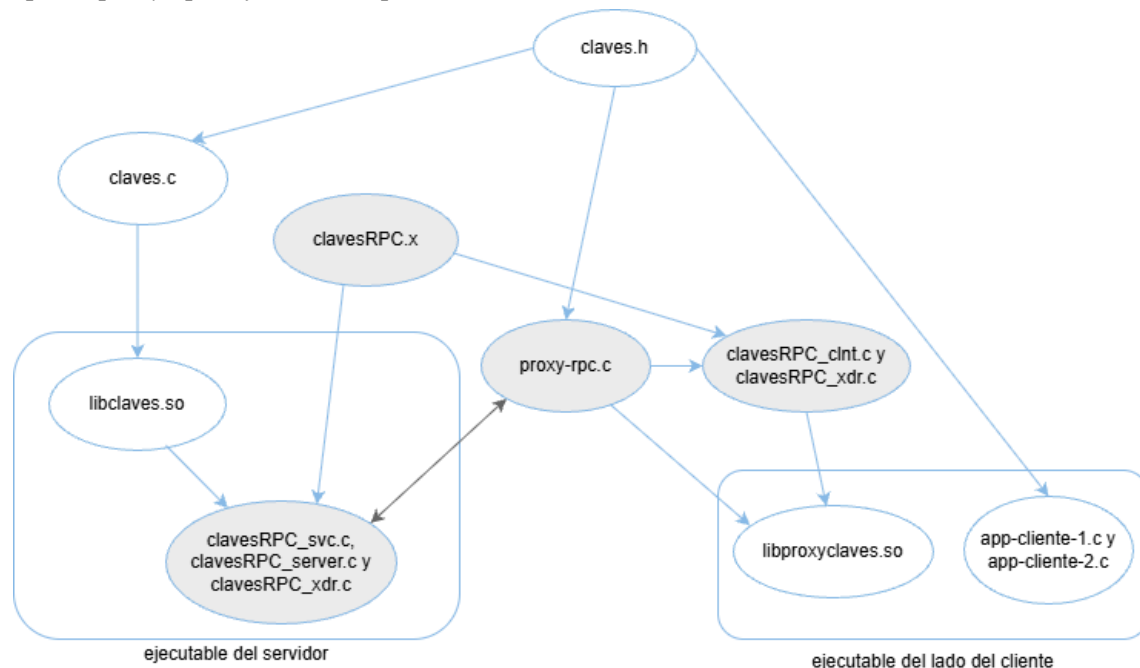
Grupo 85, Ingeniería Informática

Diseño realizado

Como RPC solo se nos permite pasar un argumento y devolver un valor, hemos diseñado estructuras personalizadas (*struct set_value_arg*, *struct get_value_ret*, *struct key_arg*) para poder empaquetar todos los parámetros de las funciones originales.

Además, hemos tenido en cuenta que, para evitar colisiones con el *struct Paquete* original del archivo *claves.h* al compilar el servidor y el proxy, hemos nombrado a la estructura de red *PaqueteRPC* en lugar de *Paquete*.

Como podemos observar en el diagrama, el código de nuestra lógica local (*claves.h* y *claves.c*) se enlaza con las rutinas generadas automáticamente por *rpcgen* gracias a nuestros dos archivos adaptados principales: *proxy-rpc.c* y *servidor-rpc.c*.



En cuanto al diseño del proxy, este actúa como una interfaz transparente para las aplicaciones cliente. Estas aplicaciones (*app-cliente-1.c* y *app-cliente-2.c*) invocan a las funciones sin ser conscientes de que existe una comunicación en red. Además, el proxy se encarga de leer la variable de entorno *IP_TUPLAS*, establecer la conexión RPC con el servidor, y realizar una traducción de los datos: empaquetando los parámetros locales y la estructura *Paquete* a las estructuras de red generadas en el *clavesRPC.x* (como *PaqueteRPC*). También gestiona los posibles errores de red devolviendo -1 en caso de fallo.

La función del servidor es recibir peticiones remotas que llegan generado por *rpcgen*, desempaquetar las variables y revertir la traducción de *PaqueteRPC* a *Paquete*. Finalmente, invoca a las funciones originales desarrolladas en *claves.c*, encargándose de devolver el código de estado de la operación (0 o -1) de vuelta al cliente.

Estructura de archivos en la compilación

Para la generación de los ejecutables y las bibliotecas del sistema, se ha diseñado un Makefile que automatiza el flujo de trabajo. Los componentes generados son:

- **libclaves.so:** que es una biblioteca que contiene la implementación real de las operaciones. Se compila *claves.c* con el flag *-fPIC* y se enlaza para crear una biblioteca compartida. El servidor RPC utiliza esta biblioteca para ejecutar las operaciones solicitadas por los clientes.
- **Soporte RPC:** se utilizan los archivos generados automáticamente a partir de *clavesRPC.x* para gestionar la serialización de datos y el protocolo de red, como por ejemplo: *clavesRPC_xdr.o*, *clavesRPC_clnt.o*, *clavesRPC_svc.o* ...
- **libproxyclaves.so:** es la biblioteca que se enlaza con el cliente. Su función es ocultar la complejidad de las llamadas RPC al programador. Implementa las funciones definidas en *claves.h* pero, en lugar de acceder a disco, realiza las llamadas remotas al servidor utilizando la IP obtenida de la variable de entorno *IP_TUPLAS*.
- **Ejecutables finales:**
 - **servidor-rpc:** es el proceso que permanece a la escucha. Se genera enlazado el código del servidor (*servidor-rpc.o*), el soporte de red (*clavesRPC_svc.o*, *clavesRPC_xdr.o*) y la lógica local (*libclaves.so*). Se utiliza el flag *-Wl,-rpath=.* para que encuentre la biblioteca *.so* en el directorio actual.
 - **app-cliente-rpc:** el cliente de pruebas principal. Se enlaza con la biblioteca proxy para realizar las operaciones de forma remota.
 - **app-cliente-rpc-concurrente:** cliente diseñado para probar la capacidad del servidor de manejar múltiples peticiones simultáneas mediante hilos.

Forma de compilar y generar los ejecutables

El proceso para probar nuestro sistema es el siguiente:

1. Limpieza y compilación: `make clean`
`make`
2. Exportar la ruta: `export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:.`
3. Inicializamos el servidor en la terminal 1: `./servidor-rpc`
4. Configuramos y ejecutamos los clientes en la terminal 2

Podemos hacerlo de dos maneras diferentes:

- a. Definiendo variables de entorno con *export*:
`export IP_TUPLAS=<IP_SERVIDOR>`
`./app-cliente-rpc`
`./app-cliente-rpc-concurrente`
- b. Pasando los valores de las variables de entorno con *env*:
`env IP_TUPLAS=<IP_SERVIDOR> ./app-cliente-rpc`
`env IP_TUPLAS=<IP_SERVIDOR> ./app-cliente-rpc-concurrente`